

MAI 5301 - Week 9

Presentation

Daryl Nelson - USI: 1021215

Background to the problem and research

Papers tackle efficiency bottlenecks in LLM inference, one from the angle of memory management for batching (PagedAttention) and the other from the angle of computational attention throughput for long contexts (Flash-Decoding). Although they optimize different parts of the serving/inference pipeline, they share the core goal of making LLM inference faster and more scalable without changing the underlying model architectures.

Efficient Memory Management for Large Language Model Serving with PagedAttention

Woosuk Kwon^{1,*} Zhuohan Li^{1,*} Siyuan Zhuang¹ Ying Sheng^{1,2} Lianmin Zheng¹
Cody Hao Yu³ Joseph E. Gonzalez¹ Hao Zhang⁴ Ion Stoica¹

The Problem: LLMs Are Powerful but Very Expensive

What's going on?

- LLMs like GPT generate text one word at a time → slow & memory-heavy
- Serving LLMs needs many GPUs → expensive per request
- GPUs are underused → generation is sequential & memory-limited

Why this matters

- To cut cost → serve more requests at once
- Batching helps → only if GPU memory is efficient

Key takeaway

- Bottleneck isn't compute → it's memory management

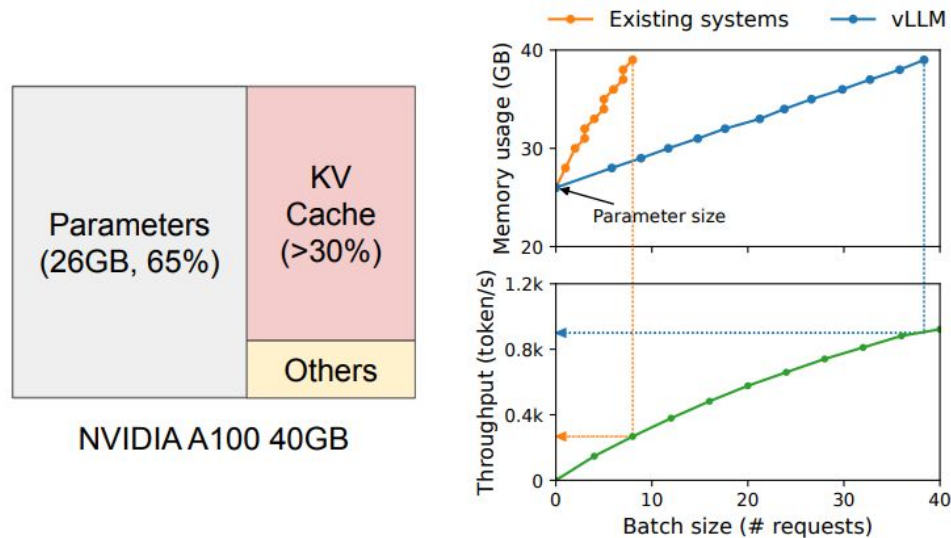


Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request. A small amount of memory (yellow) is used ephemerally for activation. *Right:* vLLM smooths out the rapid growth curve of KV cache memory seen in existing systems [31, 60], leading to a notable boost in serving throughput.

The Real Bottleneck: KV Cache Memory Waste

What is the KV cache?

- LLM stores past context in memory → Key-Value (KV) cache
- Cache grows as more tokens are generated

What existing systems do wrong

- Store each request's KV cache in one big block → causes:
 - Internal fragmentation → reserved but unused memory
 - External fragmentation → blocks don't fit together
 - No sharing → similar sequences can't reuse memory

Result

- Most KV cache memory is wasted
- Limits batch size & throughput

Key takeaway

- Wasted KV cache → fewer users per GPU

The Core Idea: PagedAttention

The insight

- OS solved memory issues with virtual memory & paging
- Paper applies this idea to LLM attention

What is PagedAttention?

- KV cache split into fixed-size blocks (pages), not one big chunk
- Blocks:
 - Allocated only when needed
 - Don't need to be contiguous
 - Can be shared across sequences

Why this works

- Removes fragmentation
- Enables memory sharing (beam search)
- Uses GPU memory efficiently

The Result: vLLM and Why It Matters

What they built

- vLLM → high-throughput LLM serving engine with PagedAttention
- Supports large models (GPT, OPT, LLaMA) → works beyond one GPU

What they achieved

- Almost zero KV cache waste
- 2 - 4× higher throughput than previous systems
- No accuracy loss
- Gains bigger for:
 - Long sequences
 - Large models
 - Advanced decoding (beam search, sampling)

Big picture

- Better memory → bigger batches → lower cost → scalable LLMs

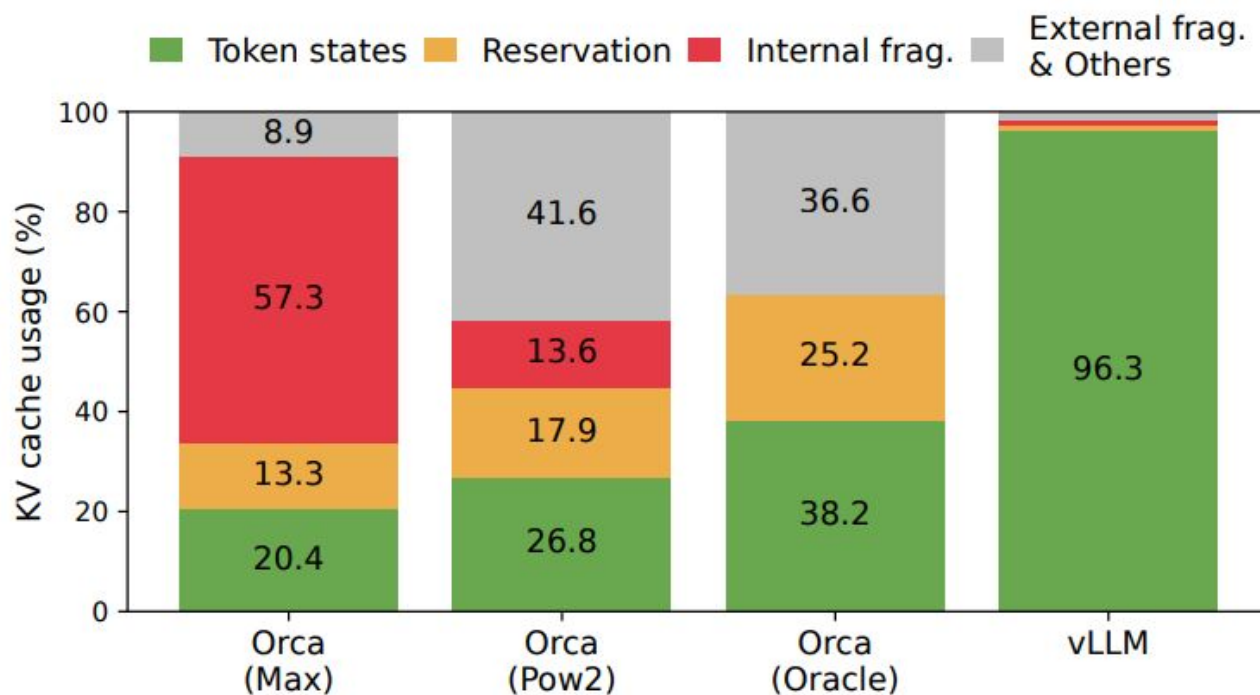


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

Background : How LLMs Generate Text

What LLMs actually do

- Predict text one token at a time
- Each token depends on all previous tokens → autoregressive generation

Why Transformers matter

- Dominant LLM architecture
- Key feature: self-attention
 - Each token looks back at previous tokens
 - Picks which ones matter most
 - Uses that to predict next token

Important simplification

- Most Transformer ops are parallel & position-wise
- Attention is expensive → needs all past context

Key takeaway

- LLMs are powerful because they remember context
- But remembering context is costly

KV Cache: The Memory That Makes LLMs Work

What is the KV cache?

- Model computes Key (K) & Value (V) vectors for each token
- Vectors capture token context & meaning
- Cached to avoid recomputing

Why caching is necessary

- To generate a new token, model needs:
 - All previous K & V
 - New token's K & V
- Without cache → generation is too slow

Crucial detail

- Same word in different positions → different K & V
- Cache grows over time
- Final size unknown

Key takeaway: KV cache is essential for speed, But dynamic, unpredictable, and memory-heavy

Two Phases of LLM Serving

Phase 1: Prompt phase

- Input prompt is fully known upfront.
- All prompt tokens are processed in parallel.
- Uses efficient GPU operations.
- KV cache for prompt tokens is generated here.

Phase 2: Autoregressive generation

- Tokens are generated one by one.
- Each step depends on all previous tokens.
- Uses inefficient GPU operations.
- This phase:
 - Can't be parallelized
 - Is memory-bound
 - Dominates latency

Two Phases of LLM Serving: Fast Then Slow

Why this hurts throughput

- GPUs are built for massive parallelism.
- Autoregressive generation barely uses GPU compute power.

Key takeaway: Most serving time is spent generating tokens sequentially; slowly and inefficiently.

Batching Helps, But Memory Management Breaks It

Why batching is needed

- Serving multiple requests together:
 - Shares model weights
 - Improves GPU utilization
 - Reduces cost per request

Why batching is hard

- Requests:
 - Arrive at different times
 - Have very different prompt and output lengths
- Naive batching causes:
 - Long waiting times
 - Padding waste (unused computation & memory)

Batching Helps, But Memory Management Breaks It

Smarter batching

- Modern systems use iteration-level scheduling:
 - Add/remove requests every generation step
 - No padding needed
 - Much higher throughput

But the hidden problem

- KV cache is still stored in large contiguous memory blocks
- This causes:
 - Reserved but unused space
 - Internal fragmentation
 - External fragmentation
- Many requests can't fit, even when memory exists

Key takeaway

Even with smart batching, inefficient KV cache memory layout becomes the real limiter.

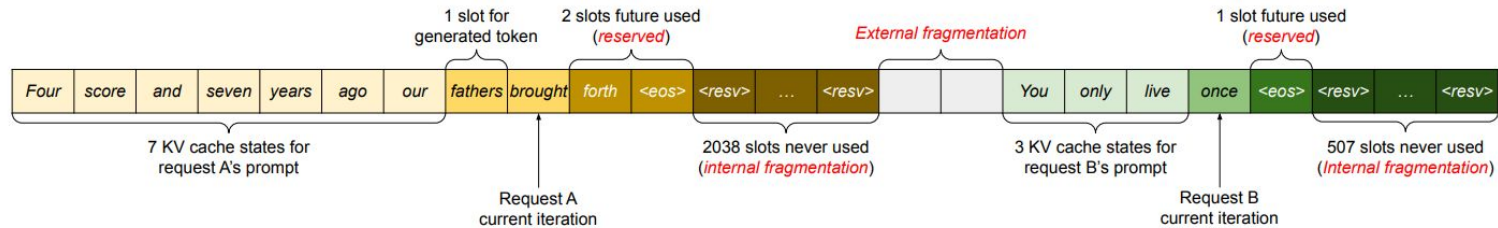


Figure 3. KV cache memory management in existing systems. Three types of memory wastes – reserved, internal fragmentation, and external fragmentation – exist that prevent other requests from fitting into the memory. The token in each memory slot represents its KV cache. Note the same tokens can have different KV cache when at different positions.

Memory Challenges: LLM Serving Is Memory-Bound

The core issue

- GPU memory limits how many requests run at once
- KV cache uses the most memory

Example

- 13B OPT model:
 - 1 token KV cache $\approx$ 800 KB
 - 2048-token request $\approx$ 1.6 GB
- 80 GB GPU $\rightarrow$ only a few dozen requests

Worsening trend

- GPUs are faster than they are bigger
- Memory, not compute, becomes the bottleneck

Key takeaway

- Efficient memory management is essential to scale LLM serving

Complex Decoding Makes Memory Sharing Hard

Why KV cache sharing matters

- Some algorithms share KV cache to save memory
- Example: multiple samples from same prompt → share prompt cache
- Beam search → can share up to 55% of KV cache
- Sharing patterns change as generation progresses

The challenge

- Memory management must adapt to decoding algorithms
- Cannot always share → outputs differ during autoregressive generation

Key takeaway

- Memory strategies must be flexible for decoding complexity

Variability in Input & Output Lengths

Problem

- Requests differ widely in prompt length and desired output length.
- KV cache grows as new tokens are generated, which may exceed available memory.

Existing solutions fail

- Allocate a fixed chunk of memory based on max sequence length.
- Leads to three types of waste:
 1. Reserved slots - memory set aside but not yet used.
 2. Internal fragmentation - memory reserved larger than needed.
 3. External fragmentation - memory allocator leaves unusable gaps.

Impact

- Real usable memory can be as low as 20% of total allocated.
- Other requests can't fit even if there's technically free memory.

Key takeaway: Fixed pre-allocation wastes massive GPU memory, limiting throughput.

Existing Solutions Are Inadequate

Why current systems fail

- Most DL frameworks require contiguous memory.
- Pre-allocated KV cache prevents memory sharing needed for complex decoding.
- Compaction (defragmentation) is too slow for performance-sensitive LLM serving.

Consequence

- Memory waste and fragmentation remain major bottlenecks.
- High GPU utilization is impossible without a new memory management strategy.

Bridge to solution

The challenges above motivate PagedAttention & vLLM, which manage KV cache in small, flexible blocks for near-zero waste.

Method: PagedAttention - Motivation & Idea

Problem: Traditional LLM attention requires contiguous memory → causes fragmentation & limits batch size.

Solution: PagedAttention, inspired by OS paging.

Key idea:

- Split KV cache into fixed-size KV blocks (like memory pages).
- KV blocks do not need to be contiguous in GPU memory.

Benefit: Flexible memory management → can store more requests efficiently.

How It Works

KV blocks & attention

- Each KV block = keys & values for a set of tokens
- Attention computed block-wise:
 - Query $\times$ key block $\rightarrow$ attention scores
 - Attention scores $\times$ value block $\rightarrow$ output
- Example: KV cache across 3 non-contiguous blocks $\rightarrow$ works seamlessly
- Enables parallelism within a block & reduces memory waste

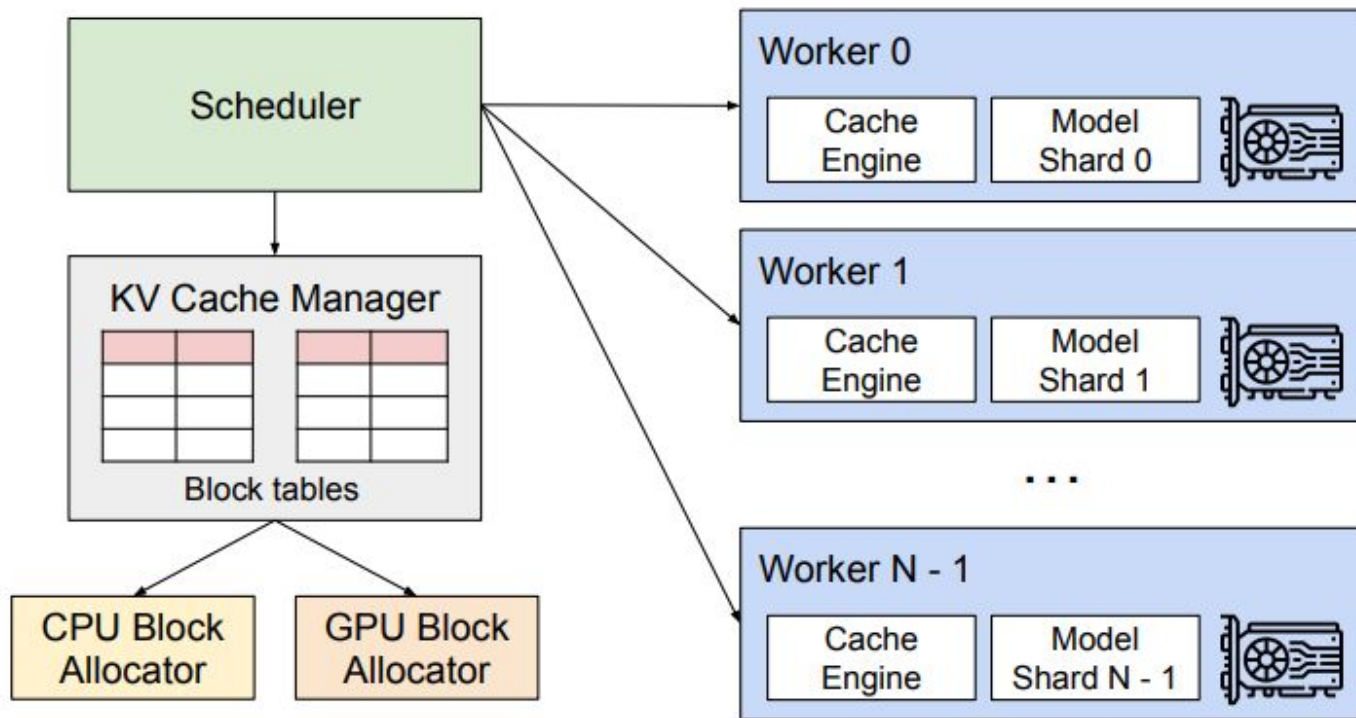


Figure 4. vLLM system overview.

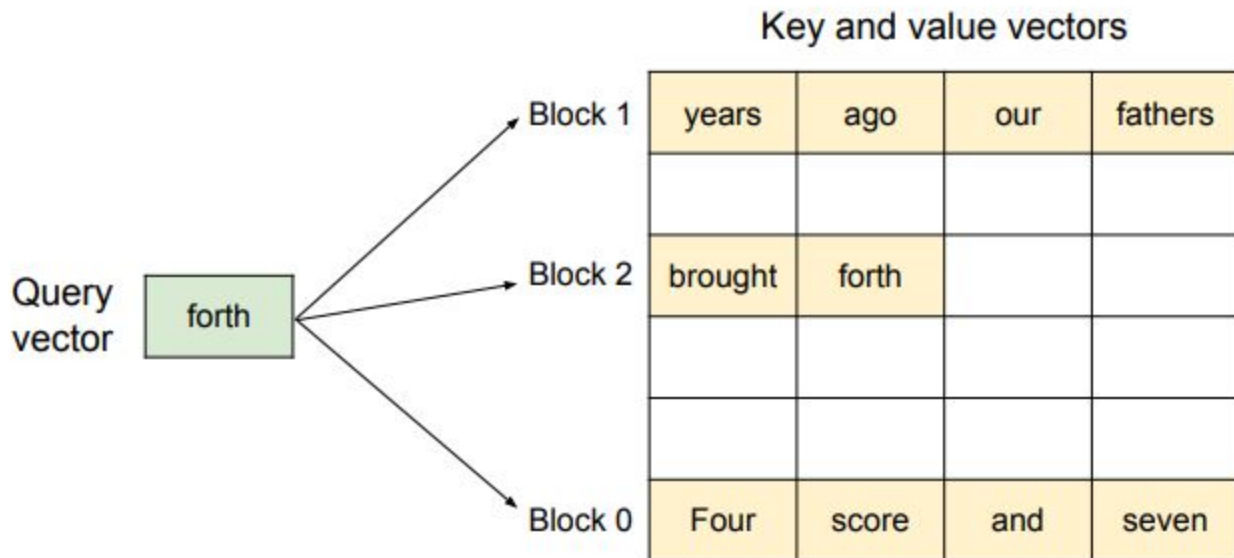


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

KV Cache Manager: OS Inspired Memory Management

KV cache as virtual memory

- Treat KV cache like virtual memory
- Logical KV blocks = virtual pages
- Physical KV blocks = actual GPU/CPU memory
- Logical → physical mapping non-contiguous & on-demand
- Last block reserved for future tokens → allows growth

Benefits & Structure

Maintains block tables mapping logical → physical blocks.

Eliminates need to pre-allocate memory for max sequence length.

Reduces internal, external, and reserved memory waste.

Allows GPU memory to store more requests → higher throughput.

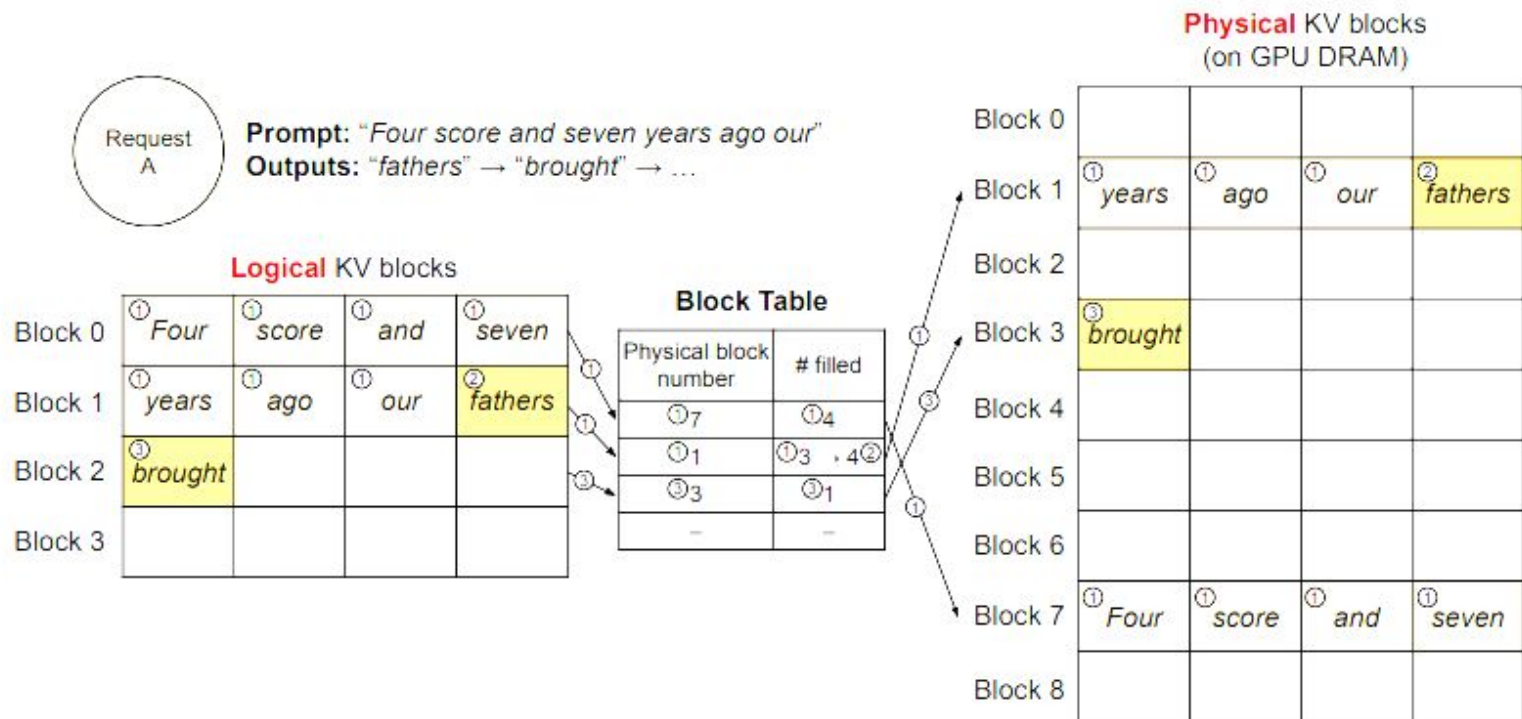


Figure 6. Block table translation in vLLM.

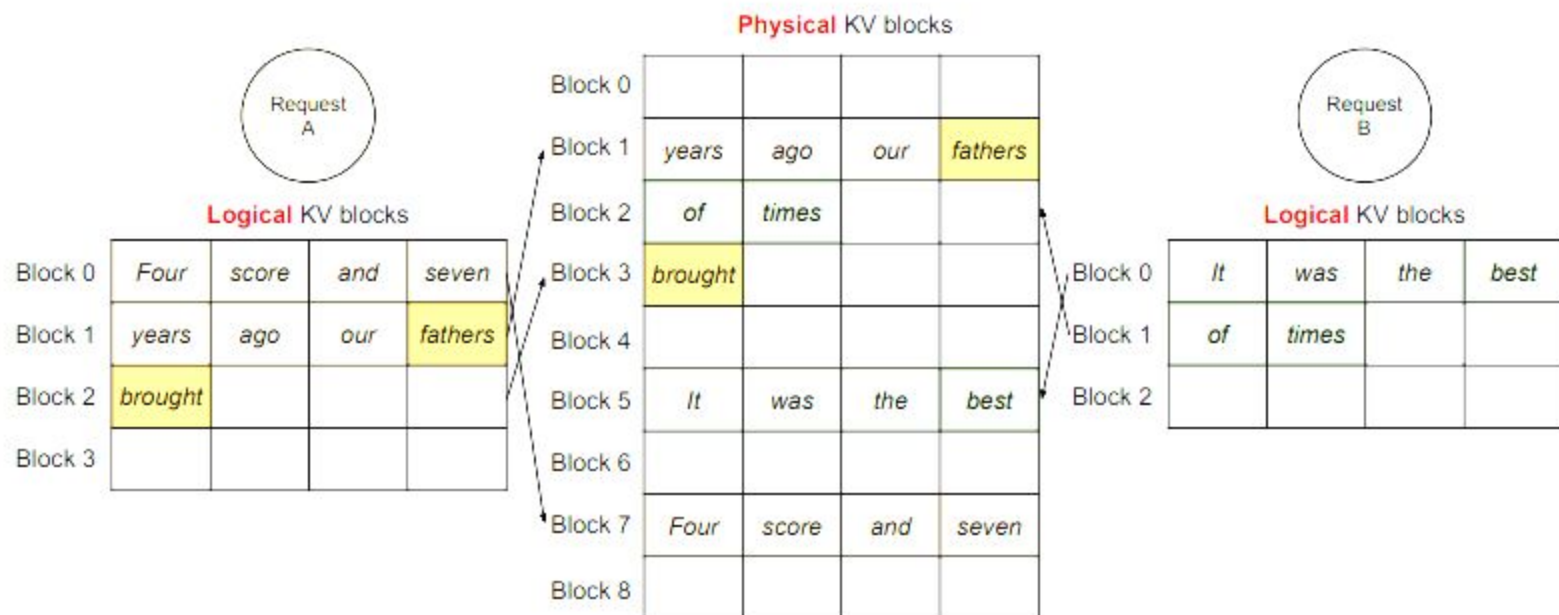


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

Decoding with PagedAttention - Step by Step Decoding

Prompt phase: Allocate only necessary blocks for prompt KV cache.

Autoregressive phase:

- Generate token → store KV in current logical block.
- If block full → allocate new physical block for next logical block.

Memory is filled left to right, minimizing waste.

Multiple Requests

Multiple sequences can share physical blocks.

Logical blocks do not need to be contiguous.

More requests fit in memory → larger batch size.

Improves hardware utilization and reduces latency.

Application to Other Decoding Scenarios: Parallel Sampling

One input → multiple sampled outputs.

Prompt KV cache can be shared across samples.

Copy-on-write for newly generated tokens only when needed.

Saves significant memory, especially for long prompts.

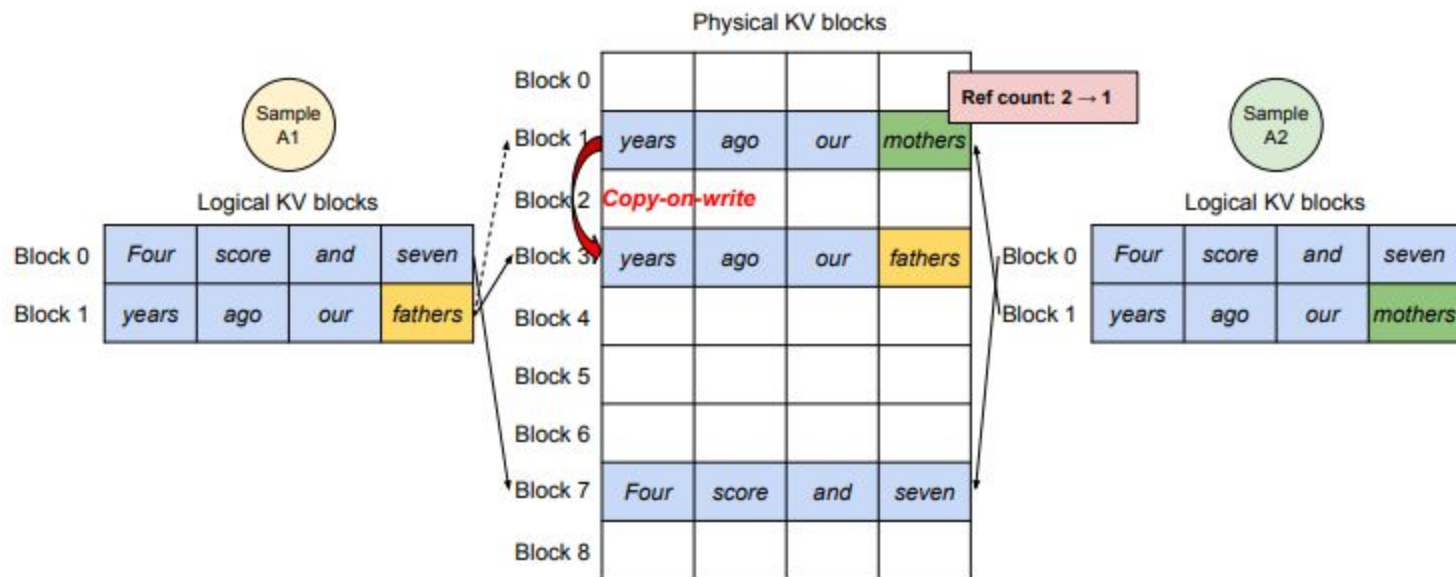


Figure 8. Parallel sampling example.

Beam Search & Shared Prefix

Beam search: multiple candidate sequences.

- vLLM shares common KV blocks across beams.
- Copy-on-write only when beams diverge.

Shared prefix (e.g., system prompt) → pre-cache KV blocks.

Supports mixed decoding methods simultaneously.

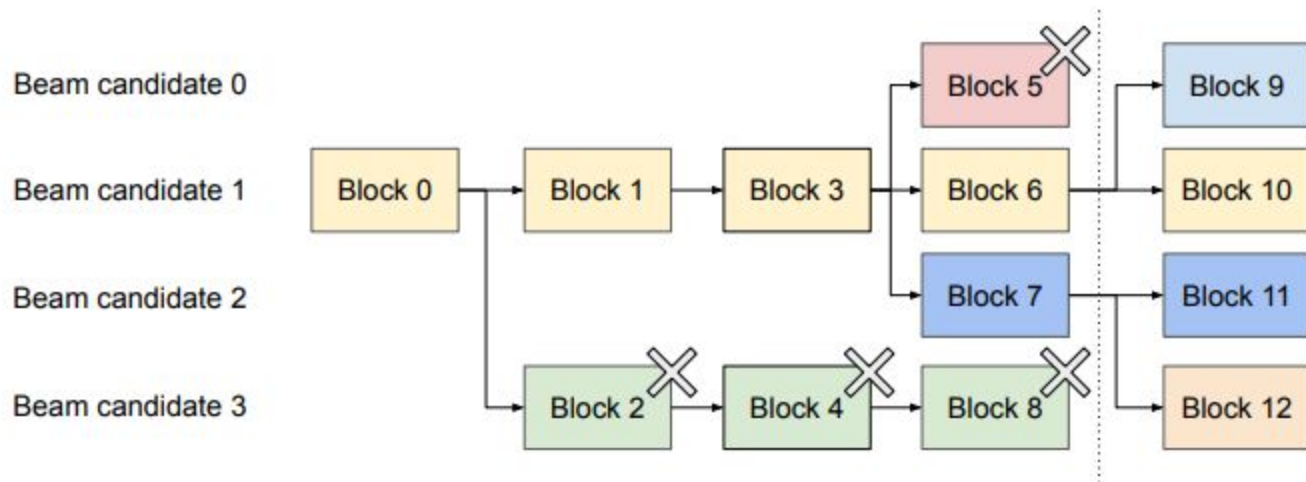


Figure 9. Beam search example.

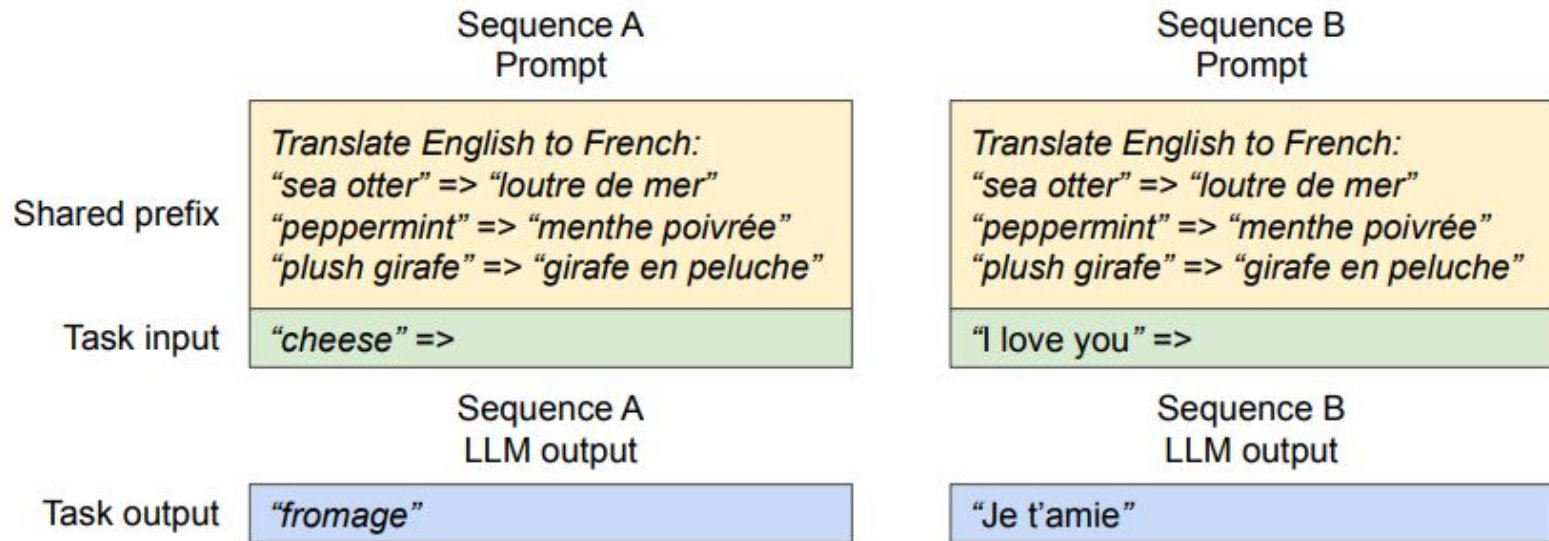


Figure 10. Shared prompt example for machine translation. The examples are adopted from [5].

Scheduling and Preemption - Handling Variable Lengths

Memory management & scheduling

- Requests have different sizes → memory can fill up
- Use first-come, first-served → fair to all requests
- Evict whole request if memory is needed
- Related requests (like beam search options) evicted together

Eviction & Recovery

Recovering evicted KV blocks

- Swapping → move blocks to CPU memory temporarily
- Recomputation → regenerate blocks when needed
- Both methods are efficient and don't stop the system

Distributed Execution -Model Parallelism

Using multiple GPUs

- Big LLMs don't fit on one GPU → split across GPUs
- Each GPU handles part of the model (attention heads)
- KV cache manager is centralized → all GPUs share memory info

Distributed Decoding

Scheduler sends to all GPUs: input tokens + block tables

GPU:

- Read KV cache using block table
- Compute attention in parallel
- Synchronize results automatically (all-reduce)

Central mapping makes memory management simple

Implementation - vLLM System Overview

vLLM System Overview

- Frontend: FastAPI + OpenAI-style API
 - Users can set sequence length, beam width, sampling
- Engine: GPU-based inference
 - 8.5K Python lines → scheduler & memory manager
 - 2K C++/CUDA lines → PagedAttention optimization
- Models supported: GPT, OPT, LLaMA (via PyTorch + Transformers)
- Distributed execution: GPUs communicate using NCCL

Kernel-Level Optimizations (1/2)

PagedAttention GPU Optimizations

- Non-contiguous memory → needs custom GPU kernels
- Fused reshape & block write → combine steps to save time
- Fused block read & attention → read KV blocks and compute attention in one pass
- Coalesced memory access → assign GPU warp per block for efficiency
- Works with different sequence lengths in the same batch

Kernel-Level Optimizations (2/2)

Fused Block Copy

- Copy-on-write can touch non-contiguous blocks
- Batch multiple copies in a single GPU call
- Goal: reduce memory overhead → higher throughput & GPU efficiency

Supporting Various Decoding Algorithms

vLLM Decoding API

- 3 key operations:
 - Fork → start a new sequence from an existing one
 - Append → add a token to a sequence
 - Free → delete a sequence

Examples:

- Parallel sampling: fork → append tokens → free finished sequences
- Beam search / shared prefix: same fork/append/free strategy
- Extensible: new decoding algorithms can use these methods

Evaluation - Experimental Setup

Models & Servers

- Models: OPT (13B, 66B, 175B), LLaMA 13B
- GPUs: NVIDIA A100 on Google Cloud (1–8 GPUs depending on model)

Workloads

- ShareGPT: long conversations, high variation
- Alpaca: shorter instructions, low variation
- Requests generated with Poisson distribution → variable rates

Baselines

- FasterTransformer: latency-optimized, dynamic batching
- Orca: throughput-optimized, 3 memory strategies:
 - Oracle: knows output length perfectly
 - Pow2: reserve $\leq 2\times$ output length
 - Max: reserve max sequence length

Metric: Normalized latency: latency per token → measures throughput under high load

Basic Sampling Results

Scenario: 1 sample per request

Observation:

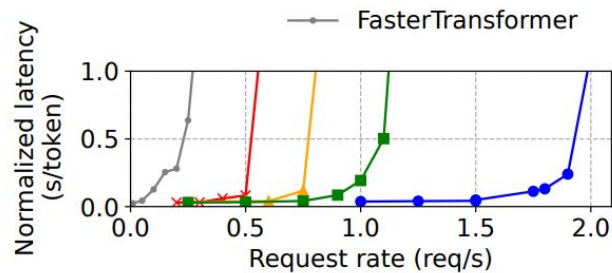
- Latency rises slowly → then spikes when system is full

vLLM Advantage:

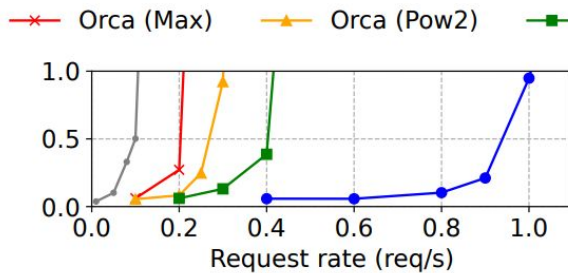
- ShareGPT: 1.7–2.7× higher request rate than Orca (Oracle), 2.7–8× than Orca (Max)
- Alpaca: smaller gain for large GPU (OPT-175B) → system becomes compute-limited
- Reason: PagedAttention → efficient memory → more requests batched

Example: OPT-13B, ShareGPT

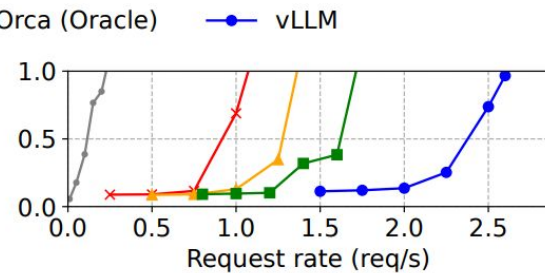
- vLLM batches 2.2× more requests than Orca (Oracle)
- vLLM batches 4.3× more requests than Orca (Max)
- FasterTransformer: up to 22× lower throughput



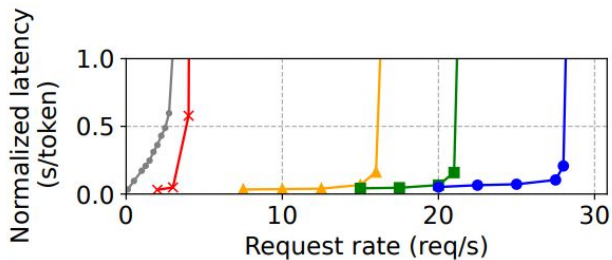
(a) OPT-13B, 1 GPU, ShareGPT



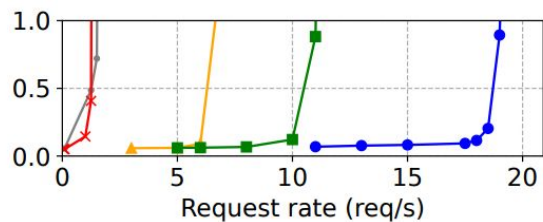
(b) OPT-66B, 4 GPUs, ShareGPT



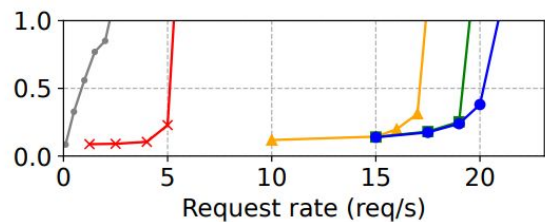
(c) OPT-175B, 8 GPUs, ShareGPT



(d) OPT-13B, 1 GPU, Alpaca

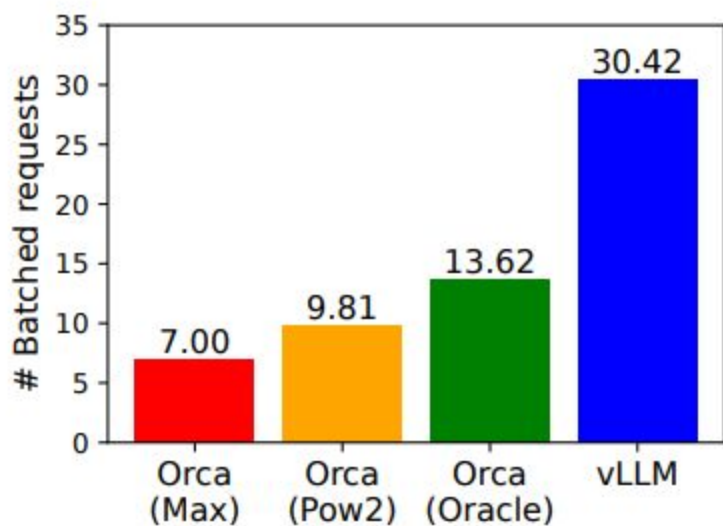


(e) OPT-66B, 4 GPUs, Alpaca

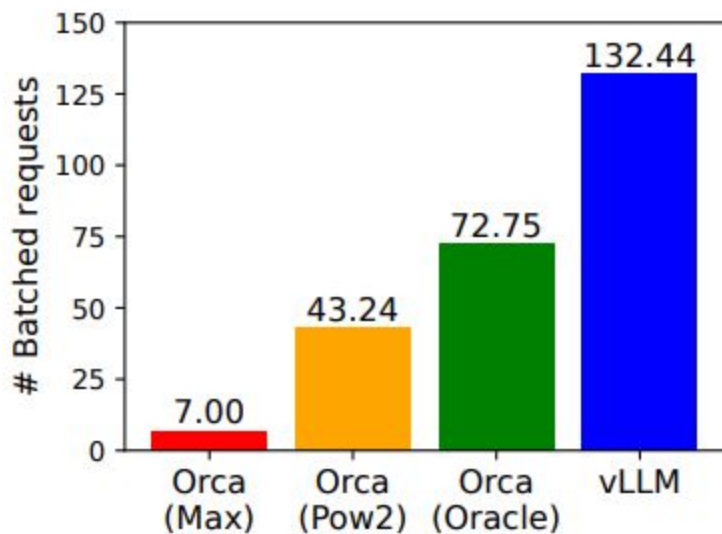


(f) OPT-175B, 8 GPUs, Alpaca

Figure 12. Single sequence generation with OPT models on the ShareGPT and Alpaca dataset



(a) ShareGPT



(b) Alpaca

Figure 13. Average number of batched requests when serving OPT-13B for the ShareGPT (2 reqs/s) and Alpaca (30 reqs/s) traces.

Parallel Sampling & Beam Search

Parallel Sampling:

- Multiple sequences share prompt KV cache
- Larger number of sequences → more improvement vs Orca

Beam Search:

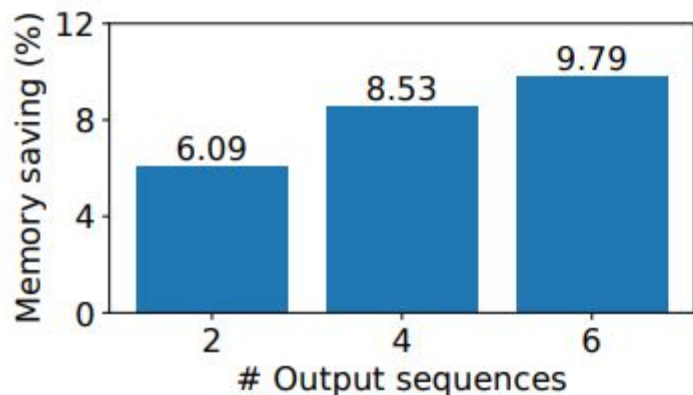
- Candidates share KV blocks dynamically
- Sharing reduces memory overhead, minimizes copy-on-write

Memory Savings:

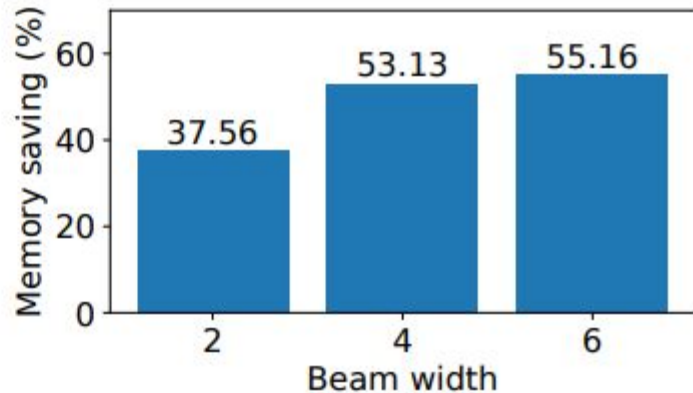
- Parallel sampling: 6–10% (Alpaca), 16–31% (ShareGPT)
- Beam search: 38–55% (Alpaca), 44–66% (ShareGPT)

Throughput Improvement:

- Beam search (width=6): 2.3× faster than Orca (Oracle)
- Basic sampling: 1.3× faster



(a) Parallel sampling



(b) Beam search

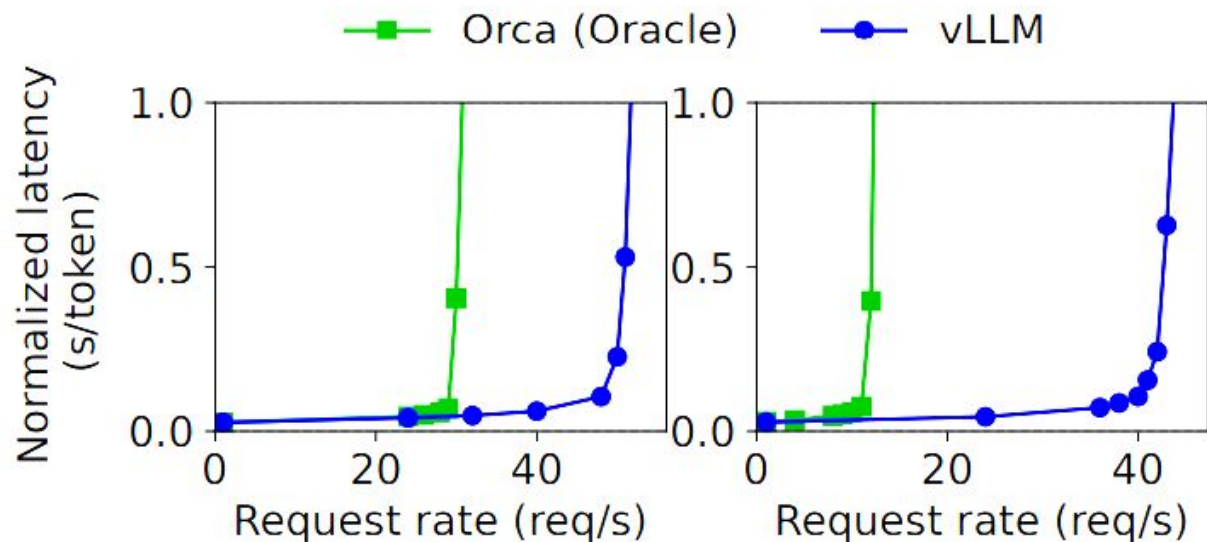
Figure 15. Average amount of memory saving from sharing KV blocks, when serving OPT-13B for the Alpaca trace.

Shared Prefix & Chatbot Workloads

- Precompute KV cache for common prompt prefixes → reuse across requests
- Results:
 - 1-shot prefix: 1.67× throughput improvement vs Orca
 - 5-shot prefix: 3.58× throughput improvement

Chatbot Workload:

- Simulated ShareGPT conversations
- Input: last 1024 tokens; output ≤ 1024 tokens
- vLLM sustains 2× higher request rates vs Orca baselines
- Reason: PagedAttention avoids memory waste from long prompts → handles large context efficiently



(a) 1-shot prefix prompt (b) 5-shot prefix prompt

Figure 16. Translation workload where the input prompts share a common prefix. The prefix includes (a) 1 example with 80 tokens or (b) 5 examples with 341 tokens.

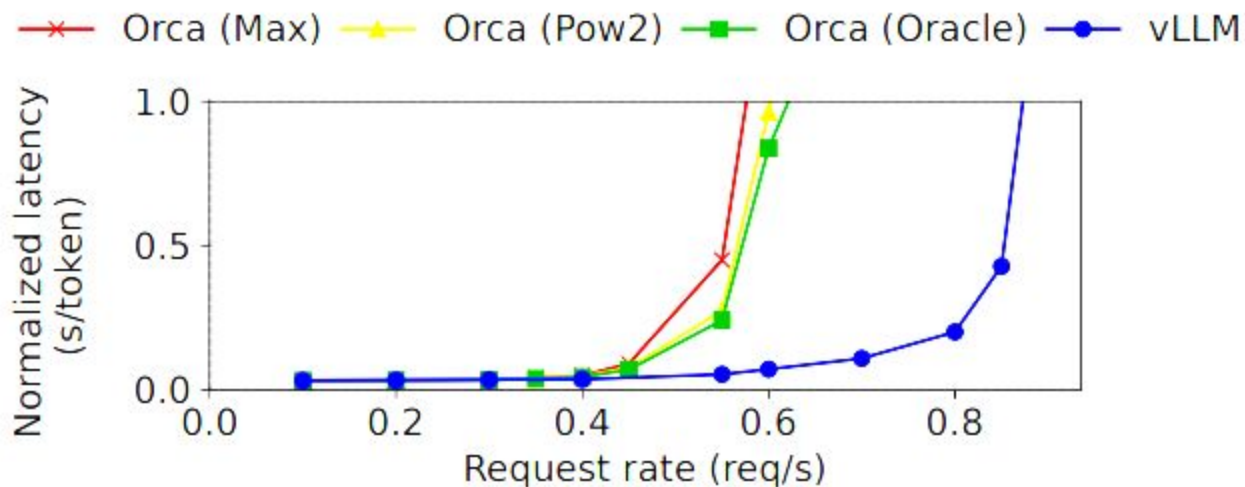
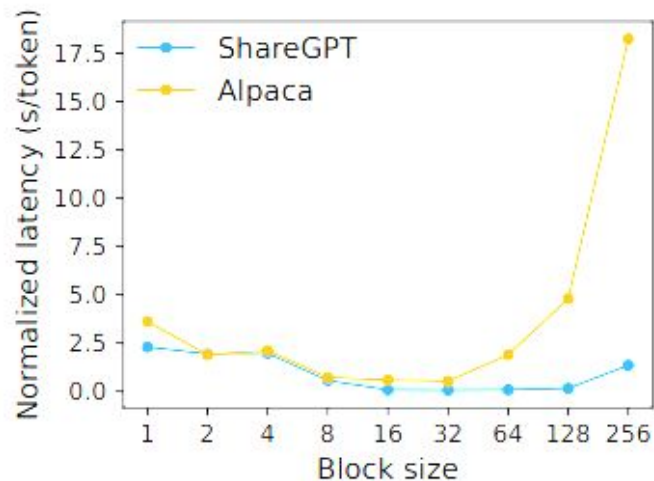
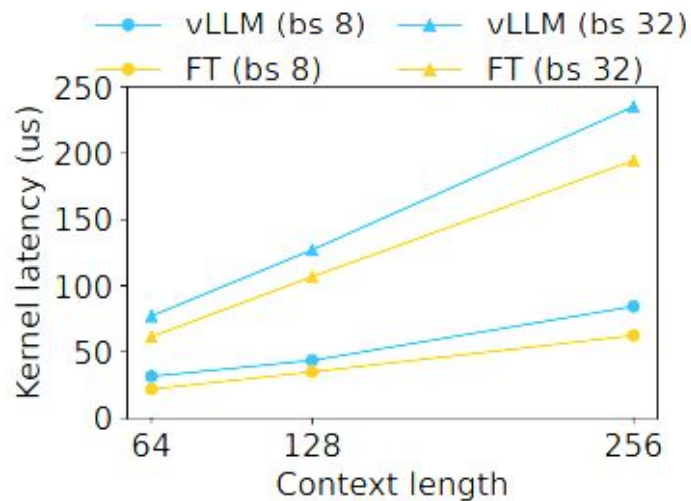


Figure 17. Performance on chatbot workload.



(a) Latency of attention kernels. (b) End-to-end latency with different block sizes.

Figure 18. Ablation experiments.

Kernel Microbenchmark

Goal: Measure GPU kernel overhead from PagedAttention

Observations:

- Dynamic block mapping introduces:
 - Extra block table accesses
 - Branching
 - Variable sequence handling
- Result: 20–26% higher latency for attention kernel vs FasterTransformer
- Impact:
 - Overhead limited to attention operator
 - Other operators (Linear, etc.) unaffected
 - Despite this, vLLM outperforms FasterTransformer end-to-end

Takeaway: Slight kernel overhead is outweighed by memory efficiency and batching benefits

Impact of Block Size

Block size effects:

- Too small: underutilized GPU parallelism
- Too large: increases internal fragmentation, reduces KV sharing

Results (Fig. 18b):

- ShareGPT: block sizes 16–128 → optimal
- Alpaca: block sizes 16–32 → best; larger sizes hurt due to short sequences

Default choice: block size = 16

- Balances GPU utilization and memory efficiency

Key insight: Block size controls performance vs memory fragmentation tradeoff

Recomputation vs Swapping (Microbenchmark)

Methods:

- Swapping: move KV blocks GPU $\leftrightarrow$ CPU
- Recomputation: regenerate KV cache on demand

Microbenchmark (Fig. 19a):

- Small block sizes $\rightarrow$ swapping overhead high (many small transfers)
- Recomputation overhead stable across block sizes
- Medium block sizes (16–64) $\rightarrow$ comparable performance

Takeaway:

- Small blocks: recomputation preferred
- Large blocks: swapping more efficient
- Recomputation never >20% slower than swapping

Recomputation vs Swapping (End-to-End Performance)

This experiment evaluates the end-to-end performance of vLLM when using recomputation or swapping as the recovery mechanism for evicted KV cache blocks, using the OPT-13B model on the ShareGPT workload under a fixed request rate.

The results show that for medium block sizes (16–64 tokens), recomputation and swapping achieve similar normalized end-to-end latency, indicating that both recovery mechanisms are effective in this regime.

When the block size becomes very large, swapping provides better performance because larger blocks reduce the overhead of CPU–GPU data transfers, making memory movement more efficient than recomputing KV states.

Because vLLM supports both recomputation and swapping, it can dynamically choose the more efficient recovery strategy depending on the block size and workload characteristics.

Overall, vLLM maintains high throughput by adapting its memory recovery strategy to the block size, rather than relying on a single fixed eviction mechanism.

Applying Virtual Memory & Paging Beyond LLMs

vLLM's virtual memory and paging techniques are effective because LLM serving is both memory-bound and highly dynamic, with unknown output lengths and continuously growing KV cache requirements.

These techniques are not universally beneficial for all GPU workloads. In DNN training, tensor shapes are typically static and known ahead of time, allowing memory allocation to be optimized offline without the need for dynamic paging.

Similarly, in non-LLM inference workloads, performance is often compute-bound rather than memory-bound, meaning that improving memory efficiency does not translate into higher throughput.

In such workloads, introducing memory indirection and non-contiguous block layouts can actually degrade performance due to additional access overhead.

The key insight is that vLLM's paging-based memory management is best suited for GPU workloads that are dynamic, memory-constrained, and require flexible allocation—properties that strongly characterize LLM serving.

LLM-Specific Optimizations

vLLM adapts the classical ideas of virtual memory and paging by incorporating semantics that are specific to large language model serving workloads, rather than applying operating-system mechanisms directly.

First, vLLM adopts an all-or-nothing swap-out policy, where all key–value (KV) cache blocks belonging to a single request are evicted together. This design leverages the fact that generating the next token for a request requires access to *all* of its previous token states, making partial eviction ineffective and unnecessary.

Second, vLLM introduces recomputation as a recovery mechanism for evicted KV cache blocks. Instead of swapping evicted blocks back from CPU memory, vLLM can recompute the required KV states by reprocessing earlier tokens, which is feasible in LLM inference but not possible in traditional operating systems where memory contents cannot be regenerated.

Finally, vLLM mitigates the performance overhead introduced by memory indirection through fused GPU kernels. These kernels combine KV block reads and writes directly with attention computation, reducing the cost of non-contiguous memory access and preserving high GPU efficiency.

Overall, vLLM demonstrates that virtual memory and paging can be highly effective when they are redesigned around the unique execution and dependency characteristics of LLM inference, achieving high throughput without sacrificing performance.

Conclusion

This work introduces PagedAttention, a new attention mechanism that enables non-contiguous KV cache storage, and vLLM, a high-throughput LLM serving system built on this idea.

By adapting operating system concepts such as virtual memory and copy-on-write to LLM serving, vLLM efficiently manages KV cache and supports advanced decoding algorithms.

Extensive experiments demonstrate that vLLM achieves 2 - 4x throughput improvements over state-of-the-art systems, establishing memory-efficient attention as a key enabler for scalable LLM serving.

Flash-Decoding for long-context inference

Tri Dao and Daniel Haziza and Francisco Massa and Grigory Sizov

Motivation

- Large Language Models are powerful but very expensive to run
- Even small inefficiencies matter when:
 - Millions or billions of users
 - Many tokens generated per request
- Inference (generation) is the main cost, not training

The paper focuses on making generation faster and cheaper, especially for long contexts.

How LLMs Generate Text

What “Decoding” Means

- LLMs generate text one token at a time
- For every new token:
 - The model looks at all previous tokens
 - Decides which ones are relevant

This happens through attention:

$$\text{softmax}(Q \times K^T) \times V$$

Even with caching:

- Keys & values from past tokens are reused
- But they still must be read every time

Attention is unavoidable and expensive.

Queries



Output



Values



Keys

Why Long Contexts Are a Problem

Attention Gets Slower as Context Grows

- If the context is:
 - 512 tokens → manageable
 - 32k–64k tokens → very expensive
- Every new token must read all past keys and values

Important detail:

- The cost is not math
- The cost is memory access

GPUs are fast at computing, slow at moving data.

Why Training Optimizations Don't Work for Decoding

FlashAttention Was Designed for Training

During training:

- Many queries at once
- Large batches
- Easy to parallelize work

During decoding:

- Query length = 1
- Batch size often = 1
- Especially true for long contexts (memory limits)

On an A100 GPU:

- 108 compute units (SMs)
- Batch size 1 → most of the GPU is idle

FlashAttention is memory-efficient, but cannot use the GPU well during decoding.

Existing Alternatives (And Their Tradeoffs)

Three Common Approaches

1. PyTorch attention
 - Simple
 - Very slow
 - Writes many intermediate tensors
2. FlashAttention v2
 - Memory-efficient
 - Poor GPU utilization in decoding
3. FasterTransformer
 - Uses matrix multiplication
 - Good GPU usage
 - Heavy memory traffic

Key Insight of Flash-Decoding

Where the Parallelism Actually Is

Even in batch size 1:

- The context length is huge
- Thousands or tens of thousands of keys & values

Flash-Decoding asks:

“Why not parallelize over the past tokens themselves?”

New parallelization dimension: keys / values sequence length

This is the core idea.

High-Level Intuition

Reading a Long Book

- One question (the query)
- A very long book (the context)

Instead of:

- One person reading the entire book slowly

Do this:

- Split the book into chapters
- Many people read chapters in parallel
- Each person gives a partial answer
- Combine answers carefully

Faster, but still correct.

Flash-Decoding: Step 1

Split the Key/Value Cache

- The long KV cache is split into chunks
- No data is copied
- Just different *views* of the same memory

Important:

- This step has no GPU cost
- It's just bookkeeping

We now have many independent pieces of work.

Flash-Decoding: Step 2

Parallel Attention on Each Chunk

For each chunk:

- Compute attention between:
 - The single query
 - That chunk's keys & values
- Use FlashAttention-style kernels

Each chunk produces:

- A partial attention output
- One extra number:
 - log-sum-exp (softmax scaling info)

Many chunks → many GPU cores working.

Flash-Decoding: Step 3

Combine Partial Results Correctly

Problem:

- Each chunk normalized its attention locally

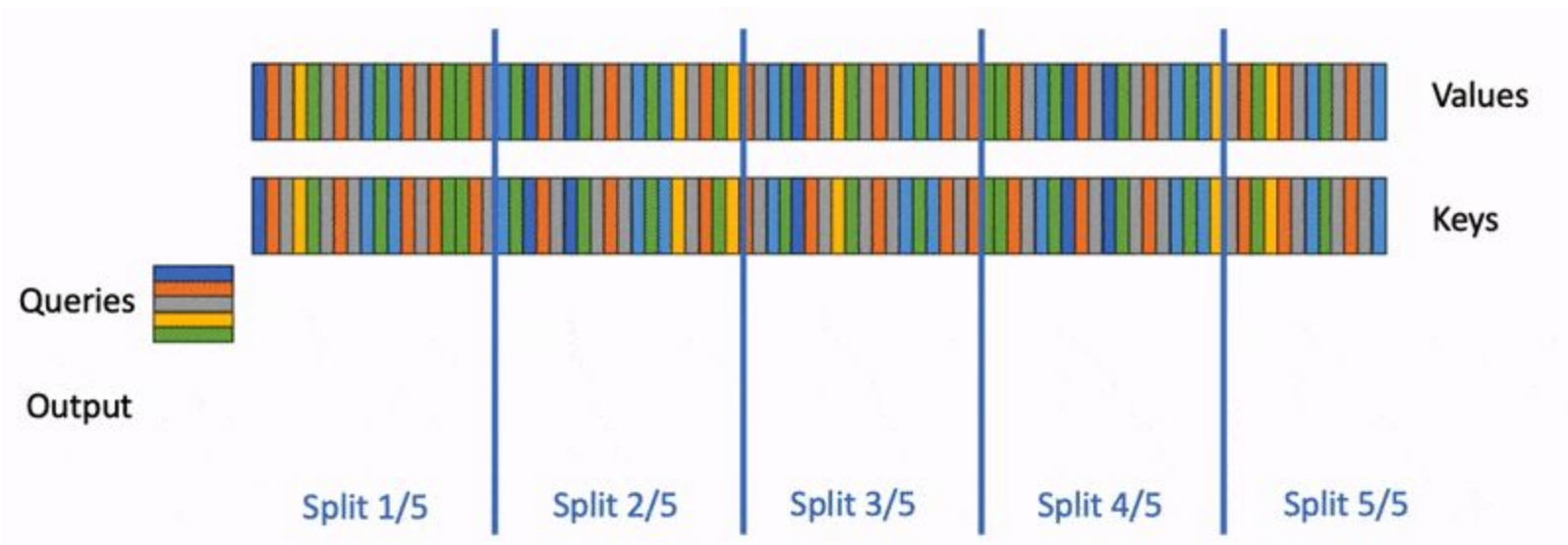
Solution:

- Use log-sum-exp values
- Rescale and merge all partial outputs

Why this works:

- Softmax can be computed iteratively
- No approximation
- Exact same result as full attention

Fast and correct.



Why This Fully Utilizes the GPU

What Changed

- Work is now split across:
 - Sequence length
 - Not just batch or queries
- Even batch size 1 can:
 - Fill all GPU cores
 - Maximize memory bandwidth

Result:

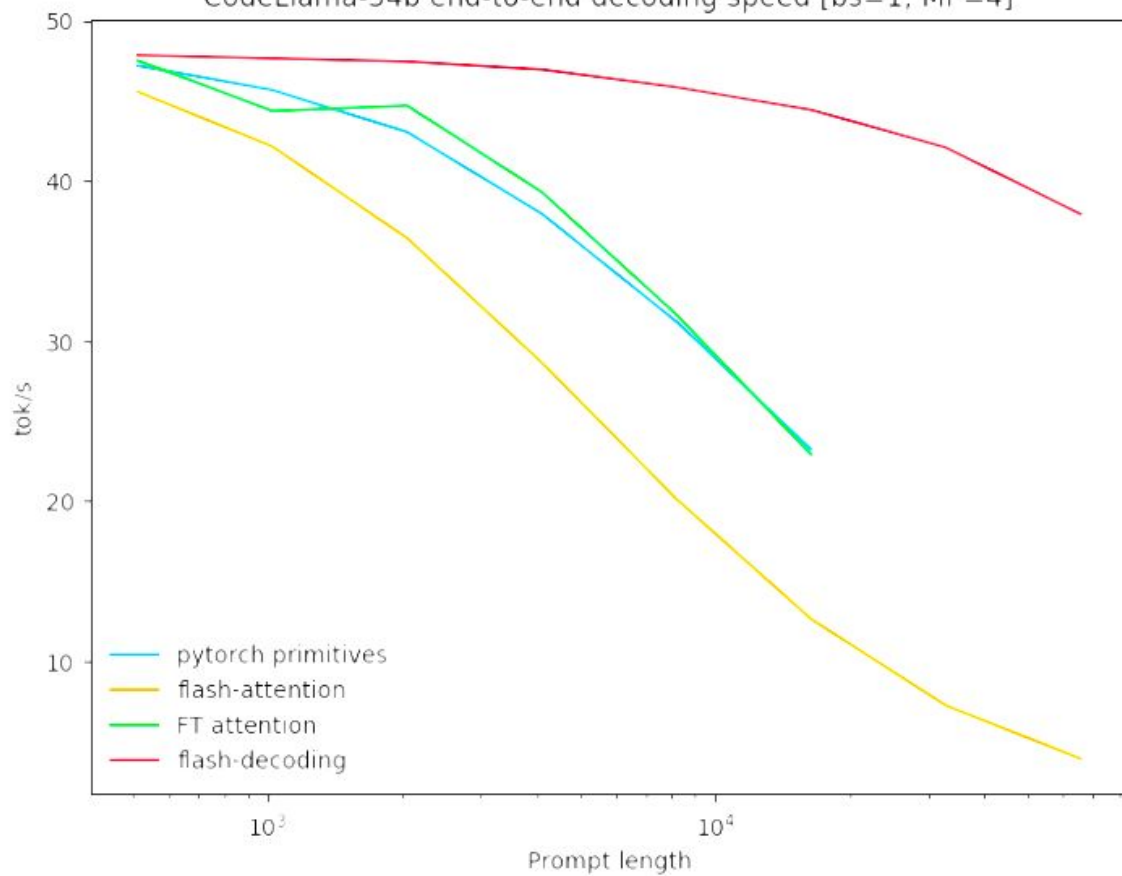
- Attention runtime becomes almost constant
- Long contexts stop being a bottleneck

End-to-End Benchmarks (CodeLLaMA 34B)

What They Measured

- Tokens per second during decoding
- Context lengths: 512 $\rightarrow$ 64k
- Compared:
 - PyTorch
 - FlashAttention
 - FasterTransformer
 - Flash-Decoding
 - Memory bandwidth upper bound

CodeLlama-34b end-to-end decoding speed [bs=1, MP=4]



Benchmark Results Explained

What Happens as Context Grows

- Small contexts:
 - All methods similar
- Large contexts:
 - Most methods slow down a lot
 - Flash-Decoding stays fast

Key result:

- Up to 8× faster decoding
- Performance close to hardware limits

Attention is no longer the bottleneck.

Micro-Benchmarks: Attention Only

Zooming In

- Measured only multi-head attention
- Same dimensions as real inference
- Batch size = 1

Results:

- Flash-Decoding attention time:
 - Nearly constant up to 64k tokens
- Attention itself:
 - Up to 50× faster than FlashAttention

Explains the big end-to-end gains.

Setting \ Algorithm	PyTorch Eager	Flash-Attention v2.0.9	Flash-Decoding
B=256, seqlen=256	3058.6	390.5	63.4
B=128, seqlen=512	3151.4	366.3	67.7
B=64, seqlen=1024	3160.4	364.8	77.7
B=32, seqlen=2048	3158.3	352	58.5
B=16, seqlen=4096	3157	401.7	57
B=8, seqlen=8192	3173.1	529.2	56.4
B=4, seqlen=16384	3223	582.7	58.2
B=2, seqlen=32768	3224.1	1156.1	60.3
B=1, seqlen=65536	1335.6	2300.6	64.4
B=1, seqlen=131072	2664	4592.2	106.6

Practical Availability

You Can Use This Today

- FlashAttention $\geq$ v2.2
- xFormers $\geq$ v0.0.22
 - Automatically selects Flash-Decoding when beneficial
- Includes:
 - Full LLaMA / CodeLLaMA examples
 - Minimal, readable decoding code

Final Takeaways

What Flash-Decoding Achieves

- Makes long-context decoding practical
- Keeps exact model behavior
- Fully utilizes GPUs even at batch size 1
- Reduces cost and latency of real deployments

Core idea to remember:

Parallelize attention over past tokens, not just batches or queries.